SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

Student Name / Reg No: R Kishan (192525378)

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I ***R Kishan (192525378)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

ANSWERS - SCENARIO BASED ASSIGNMENT

Artificial Intelligence (CSA17) | CO2 | Assessment Tool 2

Question 1: Emergency Medicine Delivery Drone (Greedy Best-First Search vs A*)

Dynamic flood region; heuristic = straight-line distance; partial real-time updates on blocked paths; variable terrain/weather cost; urgent delivery with minimum delay and risk.

Problem formulation. The region is modelled as a weighted graph/grid. State = drone location; actions = move to a neighbouring cell/waypoint; step cost c(n,n') = distance x terrain factor x weather factor (+ risk penalty); h(n) = straight-line distance to the hospital/relief camp; blocked paths are edges whose cost becomes infinity when an update arrives.

(i) Behaviour of Greedy Best-First Search, strengths and weaknesses under uncertainty

Greedy Best-First Search (GBFS) evaluates nodes only by **f(n) = h(n)**. The drone always expands the waypoint that *looks* nearest to the destination in a straight line, so it flies almost directly toward the target and ignores the cost already spent (g) and the weather/terrain cost of what lies ahead.

- **Strengths:** very fast decisions, very few node expansions, low memory and computation (suits a small onboard processor); reacts immediately to new updates because it decides locally; good when speed matters more than path quality.
- **Weaknesses:** not optimal - may choose a short-looking route through a storm or rough terrain with a very high real cost; not complete in graphs with loops unless a closed list is kept; easily trapped in dead ends (a flooded river or blocked road lies on the straight line, so h keeps pointing into it) and must then back-track; it trusts the heuristic blindly, so under uncertainty or stale updates it can oscillate or commit to risky paths.

(ii) How A* handles the situation

A* uses **f(n) = g(n) + h(n)**, where g(n) is the actual accumulated cost (distance weighted by terrain and weather) and h(n) is the optimistic estimate of the remaining cost. g makes the drone avoid expensive/risky regions already known to be bad, while h keeps the search goal-directed. If h is admissible (never overestimates; here the straight-line distance scaled by the minimum cost per unit distance) and consistent, A* is complete and returns the least-cost route, with no node re-expanded. When a path is reported blocked, the edge cost is set to infinity and A* re-plans from the drone's current position. The balance can be tuned with Weighted A* (f = g + w.h, w > 1) to trade a bounded loss in optimality for speed.

(iii) Impact of heuristic accuracy

Heuristic quality	Greedy Best-First	A*
Perfect / very accurate	Goes almost straight to the goal; fast and good path.	Expands almost only nodes on the optimal path; optimal and very fast.
Underestimate (admissible but loose, e.g. straight line in a flooded area)	Misled into dead ends and costly detours; no guarantee on path quality; many back-tracks.	Still optimal, but more nodes are expanded (slower). With h = 0 it degrades to Dijkstra/uniform-cost.
Overestimate (inadmissible)	Behaves similarly; effect is mainly on ordering.	Faster but may return a suboptimal route (loses the optimality guarantee).
Ignores terrain/weather	Highly sensitive - decisions are based only on h, so errors go straight into the route.	Less sensitive - the exact g term corrects the heuristic's error as the drone proceeds.

Hence Greedy is far more dependent on heuristic accuracy than A*. The straight-line heuristic can be improved with terrain/weather weighting or precomputed landmark distances.

(iv) Effect of dynamic changes (blocked paths)

- **Greedy:** a blocked path makes the chosen frontier node invalid; the drone is quickly re-directed but often only after flying into a dead end, wasting time and battery. Re-decision is cheap, but route quality is poor and loops are possible.

- **A*:** the computed optimal plan may become invalid, so a re-plan is needed. Re-planning from scratch costs time and the new route can differ greatly; with frequent partial updates it may re-plan too often. Incremental variants (D* Lite, LPA*) reuse earlier search effort and repair only the affected part of the plan.
- Overall, dynamic changes increase expansions, latency and risk for both; stale or partial information means the plan must be re-validated at every update.

(v) Recommendation

A* with an admissible, terrain/weather-aware heuristic, run as a replanning algorithm (D* Lite / anytime Weighted A*) is the most suitable. **Real-time:** a modest weight (w = 1.2-1.5) gives bounded sub-optimality with fast answers, and incremental replanning only repairs the changed region. **Optimality:** g(n) accounts for variable cost, so the drone follows the minimum-delay path whereas Greedy can choose a short but costly one. **Safety:** risk penalties can be added to edge costs, blocked edges become impassable, and the drone always keeps a known safe path (landing/return point). Greedy may be kept only as an emergency fallback when the battery or time budget is almost exhausted.

Question 2: Smart-City Traffic Signal Optimization (Local Search)

Hundreds of intersections; minimize waiting time, fuel consumption and congestion; huge search space; dynamic traffic; risk of local optima.

Formulation as an optimization problem and why traditional search is inefficient

- **State:** a complete timing plan s = {(cycle length, green split, offset) for every intersection i = 1..N}.
- **Objective:** minimize $f(s) = w_1.W(s) + w_2.F(s) + w_3.C(s)$, where W = total waiting time, F = fuel consumption, C = congestion index (queue length/density); evaluated by a traffic simulator or live sensors.
- **Constraints:** minimum/maximum green time, pedestrian crossing time, cycle limits, emergency-vehicle priority. **Neighbours:** change one green split or offset by a small step.

Why traditional search fails: if each of N intersections has k timing options, the space is k^N (e.g. 10^{200} for k = 10, N = 200), so BFS/DFS/A* are impossible in time and memory. There is no explicit goal state (we want the *best* state, not a path), the path to the solution is irrelevant, every evaluation is expensive, and the traffic keeps changing, so any computed solution becomes outdated.

(i) Hill Climbing and its limitations

Hill Climbing starts from the current timing plan, generates neighbouring plans (adjust one intersection's green time or offset), evaluates them and moves to the best neighbour that lowers f (steepest-ascent), or to the first better one (first-choice). It repeats until no neighbour is better. It needs only O(1) memory, is fast and can be run continuously on the live system. **Limitations:** it only accepts improvements so it gets stuck at local optima, plateaus and ridges; the result depends on the starting plan and step size; it never backtracks; and noisy sensor data may mislead it.

(ii) Local maxima, plateaus and ridges - real-world meaning

Problem	Meaning in traffic control	Example
Local optimum	A plan where changing any single signal makes things worse, but a better plan exists elsewhere.	A corridor tuned for one main road; any single change increases delays, though a completely different offset pattern is far better.
Plateau	A flat region where small changes do not change waiting time/fuel (no gradient to follow).	Off-peak traffic is light; adding 2 seconds of green changes nothing measurable, so the search wanders.
Ridge	The improving direction needs several signals to change together; single-signal moves look worse.	A 'green wave' on an arterial road: shifting only one junction's offset breaks coordination, shifting all together reduces waiting time.

(iii) How Simulated Annealing and Genetic Algorithms overcome these limitations

- **Simulated Annealing (SA):** picks a random neighbour; if it is better it is accepted, otherwise it is still accepted with probability $e^{-(\text{delta } E)/T}$. A high temperature T at the start allows exploration (escape from local

optima and plateaus); T is gradually reduced (cooling schedule) so that the search settles into a good solution. Only one solution is stored, so memory is small, and re-heating can be applied when the traffic pattern changes.

- **Genetic Algorithm (GA):** maintains a population of timing plans (chromosome = green splits and offsets of all junctions). Fitness = 1/f(s) from simulation. Selection keeps good plans, *crossover* combines well-tuned corridors/zones of different parents (this handles ridges where several variables must change together), and *mutation* introduces random changes (escapes local optima and plateaus). The population keeps diversity and can be evaluated in parallel.

(iv) Recommended approach

A **hybrid (memetic) approach**: a Genetic Algorithm (optionally with SA-style mutation) is used periodically/offline to find good base plans for each traffic regime (morning peak, evening peak, off-peak, events), followed by **Hill Climbing / low-temperature SA** for online fine-tuning every few minutes, warm-started from the current plan. **Scalability:** the network can be decomposed into zones, and GA fitness evaluations run in parallel. **Adaptability:** warm starts, population diversity and re-heating let the system react quickly to changes while avoiding local optima. If a single method must be chosen, GA is best for large scale and changing conditions, while SA is simpler and lighter.

Question 3: Autonomous Mars Rover (Online Search)

Unknown terrain, hazards (craters, rocks), limited sensing, no complete map; the rover updates its knowledge as it explores.

(i) Why an online search agent is required

An offline search agent computes a complete plan before acting and needs a known, static map. Here the map is unknown, so a complete plan cannot be computed in advance. An **online search agent** interleaves planning and acting: it chooses an action, executes it, observes new percepts and then plans again. It also knows only what is available from its current location (applicable actions, step cost after moving, goal test) and can expand only the node it physically occupies. Further reasons: the Earth-Mars communication delay (several minutes to over twenty) makes remote control impractical; hazards are irreversible (falling into a crater may end the mission); and the environment may change.

(ii) Challenges of partial observability and dynamic environments

- **Partial observability:** limited sensor range, occlusions behind rocks/slopes, noisy cameras/lidar, dust and shadows; the rover is unsure of its own pose (localization error) and of what lies ahead, so it must reason with a belief state.
- **Dynamic environment:** loose sand can slip, dust storms reduce visibility, lighting/temperature changes alter sensor reliability, wheel slip changes cost, and previously mapped areas may become outdated.
- **Further challenges:** irreversible actions (dead ends/unsafe states), balancing exploration against goal-seeking, limited battery/computation, no guarantee of finding the optimal path, and the risk of loops without memory of visited states.

(iii) How the rover builds and updates its internal model

- Maintains an **occupancy/cost map** in which each cell is unknown, free, obstacle or hazard with a probability (Bayesian/log-odds update from every sensor reading); obstacles are inflated by a safety radius.
- Maintains its pose using SLAM/odometry and stores a **result table** RESULT(s, a) = s' of transitions actually experienced, plus visited states and the learned cost estimates H(s) (as in LRTA*).
- **Cycle:** sense -> update map and localize -> plan on the current map (A*/D* Lite, unknown cells handled with a conservative or optimistic assumption) -> execute only the first few safe steps -> repeat. Hazards and a safe retreat path are always stored.

(iv) Comparison of online search with uninformed and informed search

Aspect	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online (Online DFS, LRTA*, D* Lite)
Map knowledge	Full map required	Full map required (plus heuristic)	Not required; learned while moving
Planning style	Offline, plan then act	Offline, plan then act	Interleaved planning and acting
Guidance	None; blind expansion	Heuristic h(n) towards goal	Heuristic updated from experience
Handles unknown/changing terrain	No	No (must re-run from scratch)	Yes; replans incrementally
Optimality	BFS/UCS optimal on known graph	A* optimal if h admissible	Not guaranteed; measured by competitive ratio
Efficiency/safety	Explores huge areas; unsafe on real robot	Efficient but only on known map	Efficient and safe with local sensing and retreat logic

(v) Most suitable approach

An **online informed search** - e.g. **D* Lite / LRTA*-style replanning over a continuously updated local map**, using a heuristic such as straight-line distance to the sample site. **Uncertainty:** a probabilistic map and conservative treatment of unknown cells. **Adaptability:** incremental replanning reuses earlier work whenever new obstacles are seen. **Safety:** inflated obstacle margins, risk-weighted costs, hazard reflexes (stop if the slope or wheel slip exceeds limits), always keeping a known path back to a safe state, and ground-station approval for high-risk moves. Uninformed and offline informed methods are unsuitable because they need a complete map and are slow/unsafe on the physical rover.

Question 4: University Exam Timetable Generation (CSP)

No overlapping exams for any student, limited halls, prerequisites ordering, faculty availability, special accommodations; complexity grows with courses and students.

(i) CSP formulation: variables, domains and constraints

- **Variables:** for every course c: T_c (exam time-slot) and H_c (exam hall). (Optionally one variable E_c = (slot, hall).)
- **Domains:** $D(T_c) = \{slot_1, ..., slot_m\}$ (e.g. 10 days x 2 sessions = 20 slots); $D(H_c)$ = {Hall A, Hall B, ...} (all halls with sufficient capacity).

Constraint	Type	Formal expression and explanation
No student overlap	Binary	If courses c1 and c2 share at least one student then T(c1) != T(c2). (Conflict graph edge.)
Limited halls / capacity	Binary / unary	If T(c1) = T(c2) then H(c1) != H(c2); enrolment(c) <= capacity(H(c)); total students in a slot <= total seating.
Precedence	Binary	If subject a must precede b then T(a) < T(b).
Faculty availability	Unary + binary	T(c) must be in Available(faculty of c); one faculty/invigilator cannot have two exams in the same slot.
Special accommodations	Unary	H(c) must be in accessible/separate halls for students who need them; extended-time students need long slots or a gap between their exams.
Soft preferences (optional)	Soft	Avoid two exams per day for a student, spread exams evenly, minimize total number of slots (treated as costs).

The problem is equivalent to graph colouring (courses = nodes, shared students = edges, slots = colours), which is NP-hard, hence the need for efficient strategies.

(ii) Backtracking search in this scenario

Backtracking is depth-first search that assigns one variable at a time and checks consistency with the already assigned variables. Example: assign Maths -> (Slot 1, Hall A). Next assign AI: Slot 1 conflicts because students are common with Maths, so try Slot 2, Hall B, which is consistent. If a later course (e.g. a course that must

come after both) has no legal slot left, the algorithm **backtracks** to the most recent assignment and tries another value. It finds a solution if one exists (complete), but plain backtracking is $O(d^n)$ in the worst case and repeats the same failures (thrashing).

```
BACKTRACK(assignment):
  if complete: return assignment
  v = SELECT-UNASSIGNED-VARIABLE
  for value in ORDER-DOMAIN-VALUES(v):
    if consistent(v = value):
      add (v = value) to assignment
      result = BACKTRACK(assignment)
      if result != failure: return result
      remove (v = value)
  return failure
```

(iii) How constraint propagation (forward checking) improves efficiency

After each assignment, **forward checking** removes values from the domains of unassigned neighbouring variables that conflict with it. Example: after Maths = Slot 1, Slot 1 is removed from the domain of every course sharing students with Maths, Hall A is removed for Slot 1 for other exams, and all slots up to Slot 1 are removed from courses that must come after Maths. If any variable's domain becomes empty, failure is detected immediately and the algorithm backtracks early instead of discovering the dead end many levels deeper. Stronger propagation (arc consistency, AC-3 / MAC) prunes still more, and heuristics further reduce search: **MRV** (choose the variable with fewest legal values), **degree heuristic** (most constraints), **LCV** (value that rules out fewest options).

(iv) Real-world challenges and best strategy for scalability

- **Challenges:** thousands of courses and tens of thousands of students make the conflict graph large; the problem may be over-constrained (no perfect timetable); constraints change late (hall unavailable, faculty leave, new accommodation); soft preferences and fairness conflict; privacy of accommodation data; long running time for exact methods.

Recommended strategy: (1) preprocess with arc consistency and split the conflict graph into independent components; (2) build an initial timetable with **backtracking + MAC + MRV/degree + LCV** (or DSATUR-style greedy colouring) with conflict-directed backjumping; (3) improve and repair with **min-conflicts local search** or simulated annealing, treating soft constraints as costs and relaxing the least important ones when the problem is over-constrained; (4) use incremental repair for last-minute changes instead of re-solving from scratch. This hybrid keeps hard constraints guaranteed, scales to large instances and adapts quickly to change.

Question 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

Real-time strategy game AI competing against humans; strict time limits; complex game states; very large decision tree.

(i) How Minimax models this problem

The game is modelled as a two-player, zero-sum adversarial search problem (to make it tractable, the real-time game is discretized into decision turns and treated as deterministic with a perfect-information approximation). Each **node** is a game state, each **edge** a legal move/action, MAX = the AI and MIN = the human opponent. Leaves are terminal states (win/lose/draw) or depth-limited states scored by an evaluation function. MAX nodes take the maximum of children's values and MIN nodes the minimum, assuming that the opponent plays optimally:

```
MINIMAX(s) = UTILITY(s)                 if s is terminal / depth limit
           = max MINIMAX(RESULT(s,a))   if player = MAX
           = min MINIMAX(RESULT(s,a))   if player = MIN
(max/min taken over all legal actions a)
```

The AI selects the move at the root with the highest minimax value - the best result it can guarantee against the best opponent reply.

(ii) Computational challenges of large game trees

- **Exponential growth:** with branching factor b and depth d there are about b^d nodes; e.g. b = 30, d = 10 gives about 6×10^{14} nodes. In RTS games b is far larger because many units can each take many actions.
- **Time limit:** a decision is needed within milliseconds, but the exact tree cannot be searched; **memory** for storing the tree/states is large.
- **Costly evaluation** of complex states, **horizon effect** (a threat just beyond the depth limit is missed), and **real-time/imperfect information** (fog of war, simultaneous moves) not captured by pure minimax.

(iii) How Alpha-Beta pruning improves efficiency - detailed reasoning

Alpha-Beta gives exactly the same decision as Minimax but skips branches that cannot influence it. **alpha** = best (highest) value MAX can already guarantee on the current path; **beta** = best (lowest) value MIN can already guarantee. A branch is pruned as soon as **alpha >= beta**, because the opponent (or MAX) would never allow that position to be reached.

Example: root (MAX) has three MIN children A, B, C. A has leaves 3, 12, 8, so A = 3 and alpha = 3 at the root. At B the first leaf is 2, so B <= 2; since MAX already has 3, B can never be chosen and B's remaining leaves are **pruned**. C has leaves 14, 5, 2 and its value is 2, so the root value = max(3, 2, 2) = 3 - the same as full Minimax with fewer nodes examined.

- **Complexity:** worst case $O(b^d)$ (no pruning); best case with perfect move ordering $O(b^{d/2})$, i.e. the AI can search about **twice as deep** in the same time; random ordering gives about $O(b^{3d/4})$.
- **Move ordering** is therefore crucial: try captures/attacks and previously good (killer/history) moves first, and use iterative deepening and transposition tables to reuse results.

(iv) Design of the evaluation function

Because the tree cannot be searched to the end, non-terminal leaves are scored by a weighted linear function: $\mathbf{Eval(s) = w_1f_1 + w_2f_2 + ... + w_nf_n}$, computed as (AI value - opponent value) so that it is zero-sum; a win/loss is +/- infinity.

Factor (f)	Measure	Justification	Sample weight
Army strength	Sum of unit value x health, own minus enemy	Direct indicator of battle power and ability to win fights	0.30
Economy	Resource income rate, stockpile, number of workers	Long-term capacity to build units and tech	0.20
Territory/map control	Resource points, chokepoints and vision controlled	Gives resources, safety and information advantage	0.15
Base safety	HQ health, defences, enemy units near base	Losing the base loses the game, so threats are penalized heavily	0.20
Technology/production	Upgrade level, production buildings	Future strength and tempo	0.10
Mobility/tempo	Number of available actions, army positioning	Flexibility and initiative	0.05

The function must be **fast to compute**, correlated with the chance of winning, normalized, and phase-aware (economy weighs more early, army and safety later). Weights are tuned through self-play and regression/TD learning on game outcomes.

(v) Strategy for balancing optimality and real-time performance

- **Iterative deepening alpha-beta with a time budget (anytime search):** search depth 1, 2, 3, ... until the time limit; return the best move of the last completed depth, so a good move is always available.
- **Speed-ups:** move ordering, transposition tables, killer/history heuristics, quiescence search (extend unstable positions to reduce the horizon effect), and forward pruning (beam search, late-move reductions) to cut the

branching factor.

- **Abstraction:** search over high-level macro-actions (attack, expand, defend, scout) instead of every unit command, and run the search in a background thread while the game continues.
- **Handling uncertainty/difficulty:** use determinization or a hybrid with Monte-Carlo Tree Search for hidden information; adjust depth/time to scale difficulty.

Conclusion: accept a near-optimal move found within the time limit rather than a perfect move that arrives too late. Alpha-beta with iterative deepening and a well-tuned evaluation function gives the best compromise.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand